Introduction to DL and GenAI Project

Project Report

Jan,2026 Term

Student Details :

Name: Monalika Khurana

Roll No: 23f1000559

Email: 23f1000559@ds.study.iitm.ac.in

1. Executive Summary

The project addresses the problem of music genre classification where the objective is to accurately assign audio samples from the test dataset to one of the ten predefined genres. For this task, I have implemented and evaluated three models : the Hugging Face MIT-AST (Audio Spectrogram Transformer), a custom CNN architecture, and an XGBoost classifier. These approaches allowed for a combination of deep learning and traditional machine learning techniques to be explored.The final model achieved a Kaggle leaderboard score of **0.93768.**

Through this project, I gained key insights into the deep learning workflow, particularly in handling audio datasets. As this was a first-time experience working with such data, it provided valuable understanding of audio preprocessing, feature extraction, and model training in a practical setting.

---

2. Introduction

- **Problem Statement:** The objective of this competition is to predict the correct genre label for audio files, with a focus on handling realistic and noisy conditions. The training dataset consists of songs from ten distinct genres, with each song divided into four stems. We are also provided with a noise dataset to create mashups, while the test dataset includes already mixed songs, where the task is to identify the correct genre label for each track despite the added noise.Submissions are evaluated using the Macro F1 Score across the 10 genre classes. This metric computes the F1 score independently for each genre and then averages the scores.

- **Project Objective:** As this was my first deep learning project, my primary goal was to gain hands-on experience with the practical implementation of models in audio processing. I aimed to familiarize myself with essential tools like PyTorch, Librosa, and Wandb, and deepen my understanding of audio datasets. To achieve this, I explored different model architectures, including a transformer-based pretrained architecture, a custom CNN model, and a traditional machine learning model. By experimenting with these approaches, I evaluated their performance in tackling the genre classification task while adapting to noisy inputs.

---

3. Exploratory Data Analysis (EDA):

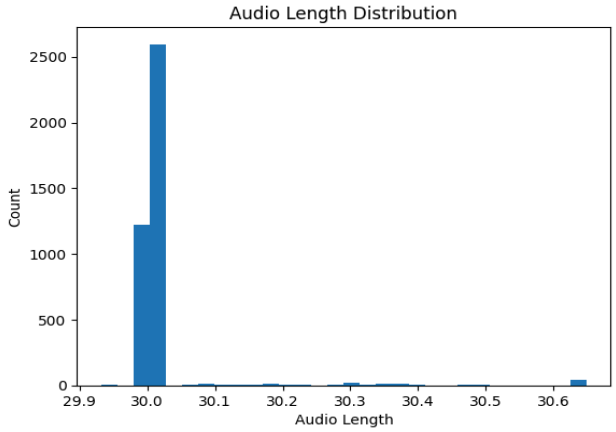
- **Dataset :** The **training dataset** is provided in **genre_stems/** and contains 10 music genres,each genre consists of 100 songs and every song is decomposed into four separate instrument stems: *drums, vocals, bass,* and *others*.

  To introduce **noise**,the dataset includes the **`ESC-50-master/`** which contains 2,000 labeled audio clips across 50 environmental sound categories and is used to simulate real-world noise conditions during mashup creation.

  The **test dataset** is located in the **`mashups/`** directory and consists of 3,020 unlabeled audio files. Each mashup is created by combining instrument stems from different songs of the same genre and adding random environmental noise at varying intensities.

  Class Distribution : The dataset is well balanced with each of the 10 genres containing 100 songs. So , equal representation of all classes which prevents bias towards any particular genre.

Audio Length Analysis : I analysed the duration of each audio stem using the Librosa library. all audio samples have a nearly uniform length, ranging approximately from 29.93 seconds to 30.65 seconds. While the test dataset has length ranging from 6s to 30s.



Sampling Rate: All audio files were found to have a sampling rate of 44,100 Hz, indicating uniformity across the dataset.

- **Data Preprocessing :**

***Resampling & Duration*:** Audio was resampled to 16 kHz and standardized to 10 seconds using padding/truncation.

***Cross-Stem Mixing*:** Each sample was created by randomly selecting stems (*bass, drums, vocals, others*) from different songs within the same genre.

***Random Stem Weighting*:** Each stem was assigned a random weight before mixing to simulate varying instrument prominence.

Time Stretching (random tempo variation).

Noise Injection using ESC-50 with random intensity.

***Normalization*:** Audio was scaled to stabilize input values.

- **A. MIT-AST (Transformer-Based Model)**

***Feature Extraction*:** Processed audio was passed through the AST feature extractor to generate inputs.

B. CNN Model

- ***Feature Representation:***
    - Converted audio into Mel Spectrograms.
    - Transformed to log scale (dB)
- ***Input Format:*** Spectrograms were treated as single-channel images for CNN processing.

C. XGBoost Model

- ***Feature Extraction:***
    - MFCCs (20 coefficients) were computed
    - Mean and standard deviation were calculated to form fixed-length feature vectors
- ***Label Encoding:*** Genre labels were converted into numerical values using a label encoder for compatibility with the model.

---

5. Modeling

5.1 Model 1: Fine-Tuned Transformer Model (AST)

Architecture

Audio Spectrogram Transformer (AST) was used as the primary deep learning model. The pretrained model from **Hugging Face** was selected and adapted for 10-class genre classification.

AST is a transformer-based model that operates on **audio spectrograms** instead of raw waveforms. It is inspired by the Vision Transformer (ViT), where spectrogram patches are treated as input tokens and processed through a stack of transformer encoder layers.

Salient Points

- **Self-Attention Mechanism:**
  The transformer uses self-attention to capture global dependencies across the audio signal, making it effective for learning long-range temporal patterns in music.

- **Patch-Based Representation:**
  Input spectrograms are divided into patches, allowing the model to process audio similarly to image data.
- **Pretraining on Large-Scale Audio Data:**
  The model is pretrained on AudioSet, enabling it to learn rich and generalizable audio representations, which significantly improves performance when fine-tuned on smaller datasets.

Fine-Tuning Strategy

- **Layer Freezing:**
  Initially, all model parameters were frozen to preserve the pretrained knowledge.
- **Selective Unfreezing**: The last 8 transformer encoder layers and the final classification head were unfrozen.
- **Loss Function:** Cross-entropy loss with label smoothing (0.1) was used to improve generalization and reduce overconfidence.
- **Optimizer:** The AdamW optimizer was used with differential learning rates:
  - Classification head: 1e-4 (faster learning)
  - Transformer layers: 1e-5 (slower, stable fine-tuning)
- **Cosine Annealing scheduler** is employed to adjust the learning rate during training. This scheduler gradually reduces the learning rate from its initial value to a minimum value of 1e-7 over the course of the training process (defined by `num_epochs`), facilitating smoother convergence.
- **Test-Time Augmentation (TTA):**
  To further improve model robustness, we implemented a **Test-Time Augmentation (TTA)** strategy, where multiple crops of the audio are extracted and classified, and the final prediction is based on the average of the logits across these crops. For each input audio file:

  Audio is randomly cropped (with 5 crops) to create multiple augmented versions.

  Each crop is processed through the model and the resulting logits are aggregated.

  The final genre prediction is determined by the average logits and selecting the genre

  with the highest avg score.

This technique allows the model to capture variations in audio that may be missed with a

single pass, improving overall prediction accuracy.

5.2 Model 2: Custom Deep Learning Model (CNN)

Architecture

A custom **Convolutional Neural Network (CNN)** was designed to classify music genres using **Mel Spectrograms** as input. The model treats spectrograms as image-like representations, where the two dimensions correspond to time and frequency.

The architecture consists of the following components:

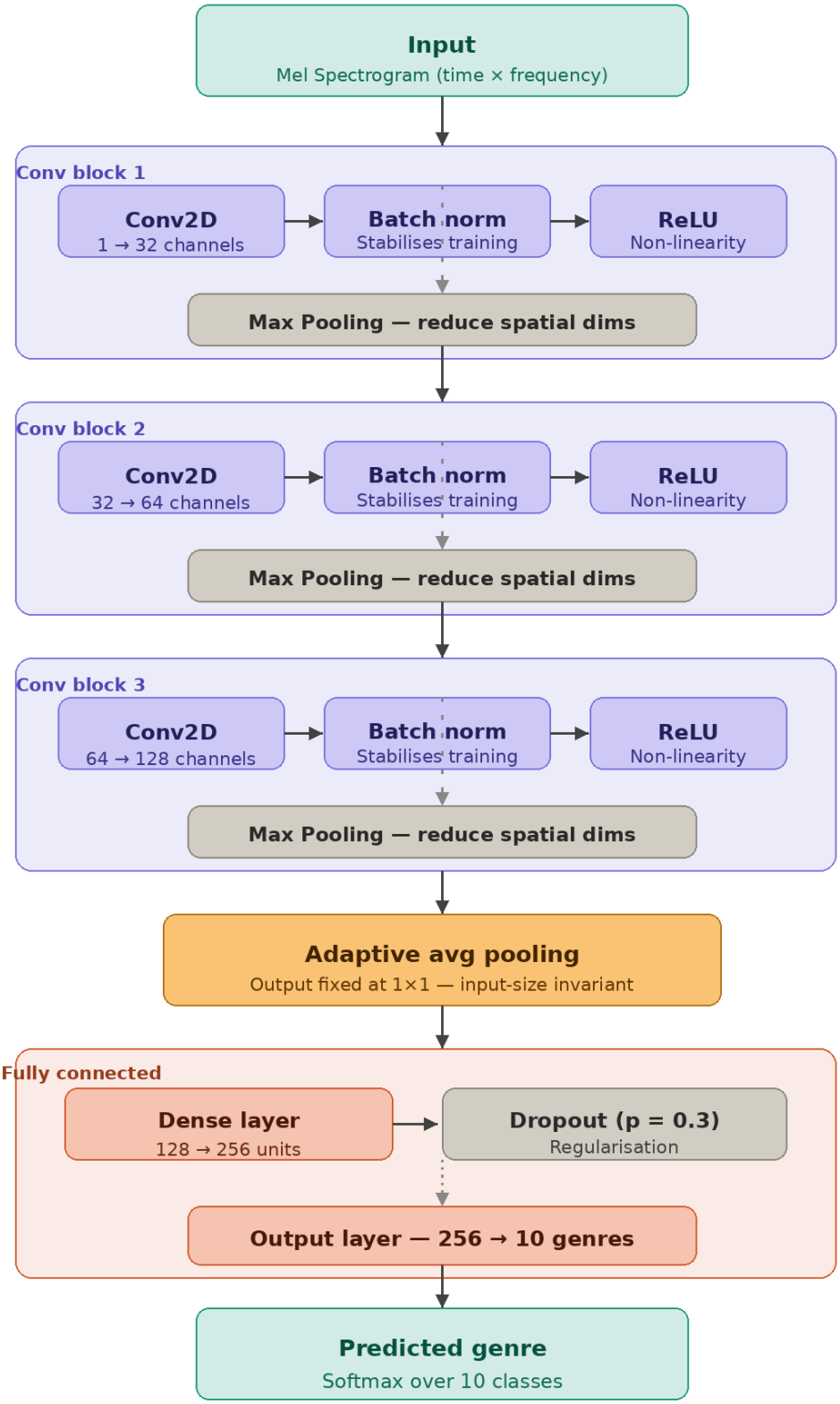
- **Three Convolutional Blocks:**
  - Conv2D → Batch Normalization → ReLU → Max Pooling
  - Channel progression: **1 → 32 → 64 → 128**
- **Pooling Layers:**
  Max pooling is applied after each convolutional block to progressively reduce spatial dimensions and capture higher-level features.
- **Adaptive Average Pooling:**
  Reduces the final feature map to a fixed size of **(1 × 1)**, making the model independent of input dimensions.
- **Fully Connected Layers:**
  - Dense layer: 128 → 256
  - Output layer: 256 → 10 (number of genres)
- **Regularization:**
  Dropout (0.3) is applied before the final classification layer to reduce overfitting.

Salient Points

- **Use of Mel Spectrograms:**
  Spectrograms transform audio into a time-frequency representation, allowing CNNs to learn spatial patterns such as harmonic structures and rhythmic variations.
- **Hierarchical Feature Learning:**
  The stacked convolutional layers enable the model to learn:
  - Low-level features (edges, frequency bands) in early layers
  - High-level patterns (rhythm, timbre structures) in deeper layers
- **Batch Normalization:**
  Helps stabilize training and improves convergence by normalizing feature distributions after each convolution.
- **Max Pooling:**
  Reduces dimensionality while retaining the most important features, making the model computationally efficient.

- **Adaptive Average Pooling:**
  Ensures a fixed-size output regardless of input variations, improving flexibility and simplifying the fully connected layers.
- **Dropout Regularization:**
  Prevents overfitting by randomly deactivating neurons during training, improving generalization on unseen data.

DIAGRAM:


Input
Mel Spectrogram (time × frequency)
Conv block 1
Conv2D
1 → 32 channels
Batch norm
Stabilises training
ReLU
Non-linearity
Max Pooling — reduce spatial dims
Conv block 2
Conv2D
32 → 64 channels
Batch norm
Stabilises training
ReLU
Non-linearity
Max Pooling — reduce spatial dims
Conv block 3
Conv2D
64 → 128 channels
Batch norm
Stabilises training
ReLU
Non-linearity
Max Pooling — reduce spatial dims
Adaptive avg pooling
Output fixed at 1×1 — input-size invariant
Fully connected
Dense layer
128 → 256 units
Dropout (p = 0.3)
Regularisation
Output layer — 256 → 10 genres
Predicted genre
Softmax over 10 classes

5.3 Model 3: Traditional Machine Learning Model (XGBoost)

Architecture

For the third approach, a traditional machine learning model, XGBoost, (Extreme Gradient Boosting) was used for music genre classification. XGBoost is an optimized gradient boosting framework that builds an ensemble of decision trees in a sequential manner, where each subsequent tree attempts to correct the errors of the previous ones.

In this project, the model was implemented using the XGBClassifier, which supports multi-class classification through a softmax-based objective function. Instead of raw audio or spectrograms, the model operates on feature vectors extracted from audio signals **(MFCC-based features).**

---

6. Performance & Comparative Analysis

Model	AST	CNN	XGBOOST
Epochs	15	15	300 estimators
Batch_size	8	32	—
Runtime	2h 42m 58s	1h 6m 23s	11m 56s
Train Loss	0.6519	0.0499	1.09408
Test Score (Macro F1)	0.93768	0.45810	0.17124

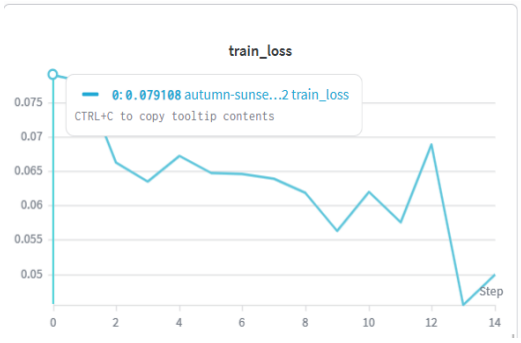
Kaggle Performance

Final Score (Macro F1): **0.93768**

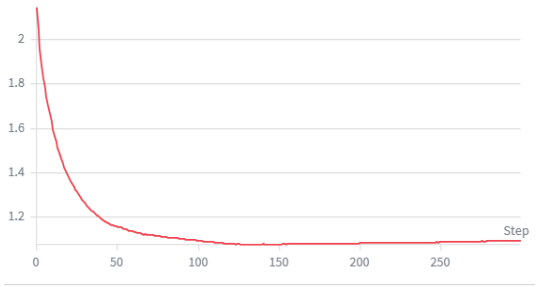
Leaderboard Rank: **118**



AST



CNN



XGBOOST

---

7. Deployment

The trained model was deployed using Hugging Face by pushing both the model and feature extractor to a repository (`m7k-run/ast-genre-classifier`) using the Hugging Face Hub API. A web interface was then created using Gradio by setting up a Gradio Space and adding the `app.py` and `requirements.txt` files for interactive inference.

Deployment Link:

https://huggingface.co/spaces/m7k-run/ast-classifier

---

8. Conclusion & Future Work

Key Learnings

This project provided hands-on experience with the complete deep learning pipeline, from data preprocessing to model evaluation. One of the most important learnings was understanding how different types of models—transformer-based, convolutional, and traditional machine learning—perform on the same task.

Working with the **Audio Spectrogram Transformer** helped in understanding the practical aspects of fine-tuning large pretrained models, including layer freezing, differential learning rates, and the importance of leveraging pretrained knowledge. Additionally, the project improved my understanding of audio data processing, including feature extraction techniques such as Mel spectrograms and MFCCs. Tools like **PyTorch**, **Librosa**, played a crucial role in building, training, and monitoring the models.

Challenges Faced

One of the major challenges was handling **audio data and preprocessing**, as this was my first experience working with such data. Understanding how to convert raw audio into meaningful representations required significant experimentation.

Another challenge was **computational limitations**, particularly while training the transformer model. Due to memory constraints, a small batch size had to be used, which increased training time.

Designing effective **data augmentation strategies** was also challenging. Finally, the **XGBoost model performance** highlighted the difficulty of relying solely on mfcc features for complex audio tasks, reinforcing the importance of representation learning in deep learning models.

Areas for Improvement

- **Advanced Architectures:**
  Exploring and studying more recent or hybrid architectures (e.g., CNN + Transformer combinations) ,Different pretrained models could help get a better score.

- **Ensemble Methods:**
  Combining predictions from multiple models (e.g., AST + resnet) could improve overall performance and robustness.

---

9. References

- Youtube Channels : CampusX, AyushDev, Krish Naik and many more
- Audio Spectrogram Transformer model Hugging Face
- Live sessions
- PyTorch Documentation
- Librosa audio processing library
- XGBoost machine learning framework